Seasonal and Weather-Based HSL City Bike Allocation

Chenyu Wu, Jizhou Wang and Jussi Honkio
Technical implementation, evaluation and reflection

1. Project motivation and scope

Our mini-project began with a practical question: where should city-bike operators allocate more bicycles or review station capacity during different months and quarters? The original canvas proposed combining journey histories, weather, maps and supervised regression into a planning website for city-bike operators and HSL transport planners. It also considered forecasts at monthly, weekly, daily and hourly horizons, weather forecasts, uncertainty and an exportable allocation-review list. [1]

The implemented project covers Helsinki and Espoo over the April-October cycling seasons of 2024 and 2025. It combines monthly station-level forecasting, retrospective daily weather analysis and historical hourly profiles. This narrower scope provides a consistent two-season comparison and interpretable outputs. The intended heatmap product evolved into a planning dashboard: it helps users explore activity and directional imbalances, but does not yet deliver short-horizon forecasts of incoming station hotspots or prescribe the precise number of extra bicycles and docks.

Data collection and backend structure

The Python downloader retrieves 14 monthly origin-destination CSVs from HSL, station metadata from an official ArcGIS service and daily weather observations from the Finnish Meteorological Institute (FMI). Journey fields include origin and destination IDs, timestamps, distance and duration. Metadata supply names, municipality, dock capacity and WGS84 coordinates. Cached non-empty downloads can be reused, reducing repeated requests. [2]

The repository reports 5,102,233 raw journeys and 4,951,245 retained after cleaning, a 97.04% retention rate. The retained totals are 2,507,175 in 2024 and 2,444,070 in 2025. The main analytical table contains 6,395 station-month rows. The metadata catalogue has 457 stations, while the web export represents 454 mapped stations appearing in the journey analysis and seven unmapped IDs. These counts describe different populations and should not be treated as interchangeable. [2,3]

Collection, cleaning, notebooks, figure generation and web-data export are separated into modules. The main runner connects downloading and aggregation; notebooks contain exploratory analysis and model comparison. This division worked well because computational work happens before the browser loads the results, and intermediate tables can be inspected independently. It also makes the technical work reproducible without requiring the presentation interface.

The central technical challenge

The project required more than fitting a regression model. We had to align several data sources, define what a station-month means, distinguish missing observations from zero activity, and ensure that predictors would actually be available when a forecast is issued. These decisions determine whether the outputs answer the planning question. A large historical dataset alone cannot reveal unserved demand, empty-station risk or the effect of expanding a station.

2. Preprocessing and weather integration

A consistent analytical unit

The cleaning function standardizes column names, converts distance and duration to numeric values, normalizes station IDs to integers and drops records missing required fields. It retains trips lasting 10 seconds to 24 hours, covering at least 10 metres, with average speed no greater than 10 metres per second and return time no earlier than departure. These are plausibility rules rather than proof that every retained journey is valid; some legitimate short trips may also be removed. [2]

For each monthly source file, departures and returns are counted separately by station and combined with an outer join. A station appearing on only one side receives zero for the other flow. Net flow is returns minus departures; total station activity is their sum. Summing activity across the network counts each journey twice because it has two endpoints. Monthly returns follow the source-file cohort, whereas the intraday pipeline uses each event's own timestamp. Cross-month attribution should therefore be audited before interpreting month-end inventory changes.

The intraday pipeline parses timestamps, distinguishes weekdays from weekends and aggregates departure and return events into their respective hours. It records midnight crossings and unmapped endpoints. Dividing by calendar weekdays or weekend days avoids confusing more operating days with greater daily intensity. These outputs are historical average hourly profiles within each month, rather than forecasts of individual future hours. Station-specific opening and closure dates would further improve the normalization.

Missingness and quality checks

Unmapped station IDs retain missing coordinates and municipality and are excluded from maps. Missing historical counterparts are excluded from model comparison instead of being treated as zero demand. However, an entirely absent station-month is not automatically a verified zero: aggregation starts from recorded events, not a complete historical operating-status panel.

The canvas called for duplicate checks and alignment of station openings and closures. The inspected cleaner does not explicitly deduplicate raw journeys, and current metadata do not provide versioned historical capacities. These remain improvements. The model also encodes unknown-city records as non-Helsinki; an explicit unknown category would be clearer. The preserved audit records are useful precisely because they make these limitations visible.

Weather processing and scope

FMI daily mean, minimum and maximum temperature, precipitation and snow depth come from Kaisaniemi and Tapiola. The repository reports 428 seasonal daily observations per weather station and complete coverage for all five parameters. UTC observation times are converted to Europe/Helsinki. Monthly temperature and rainfall measures are joined by municipality and month. The pipeline converts negative precipitation codes to zero and defines a rainy day as at least 1 mm. Integration tests cover this convention, completeness, key uniqueness and join integrity. [2,3]

Wind speed and archived weather forecasts were not implemented. One observation station per municipality also misses local variation. Most importantly, actual target-month weather is unknown when the forecast is issued. We therefore retain weather for retrospective exploration and exclude it from predictive features. This limits the original weather-based forecasting ambition, but makes the evaluation more credible.

3. Forecasting design and evaluation

A temporal benchmark

We compare a previous-year same-month baseline, Ridge regression and a decision tree. Departures and returns are modelled separately, and predicted net flow is their difference. The baseline assigns each 2025 station-month the same station's volume in the corresponding month of 2024. Across 3,170 matched April-October records, its departure MAE is 147.60 trips per station-month and WAPE is 19.23%; 27 unmatched records are excluded. [3]

Correcting feature leakage

An earlier exploratory feature used each station's full-season mean, including the target month and later months in the training season. The notebook identifies this as leakage and replaces it with the preceding calendar month from the same season. At the start of month m, month m-1 is complete and available. Training uses May-October 2024; testing uses May-October 2025. April supplies the initial lag and is not forecast by the lag models. This correction showed why information availability at prediction time must govern feature design. [4]

Each regression has ten features: the relevant prior-month count, capacity, latitude, longitude, a Helsinki indicator and five month indicators. Missing numeric metadata use training-set medians. Ridge uses training-fitted standardization and alpha = 10. The tree uses maximum depth 6, minimum leaf size 10 and random seed 42. Predictions are clipped at zero. The comparison has 2,738 training observations and a shared cohort of 2,718 test observations requiring both a lag and a same-month baseline counterpart. [4,5]

Model	Departure MAE	Departure WAPE	Return MAE	Return WAPE
Same-month baseline	141.88	17.58%	139.73	17.31%
Ridge regression	226.01	28.00%	226.52	28.06%
Decision tree	199.53	24.72%	195.63	24.23%

Table 1. Shared May-October 2025 cohort, N = 2,718. MAE: completed trips per station-month. Source: repository model summary [5].

What worked and what did not

MAE averages absolute count errors. WAPE is 100 times summed absolute errors divided by summed observed counts. The baseline has the lowest overall error on the common cohort. Matching the calendar month may preserve station scale and seasonality better than a recent-month lag, but this is an interpretation, not a demonstrated causal mechanism.

Ridge performs better on another objective: it captures an average 69.2% of the actual top-20 net-outflow stations, compared with 60.8% for the baseline across six months. Yet among the 681 low-use test observations with at most 198 departures, Ridge has 124.07% departure WAPE versus 33.83% for the baseline. This subgroup is defined retrospectively from test outcomes. Count accuracy, ranking quality and quieter-station performance must therefore be assessed separately. [4,5]

These are results from one held-out season, not evidence of long-term superiority. In particular, thousands of station-month records do not provide thousands of independent seasonal tests. More years and a separate validation period are needed before selecting a production model or claiming that one method generalizes best.

4. Findings, visualization and platform

Spatial and temporal findings

Retained journeys decline by approximately 2.5% between the two seasons, but the more useful findings concern station and time-of-day differences. Helsinki accounts for most regional trips, and high-volume station-months cluster around central activity and transit hubs. Normalized weekday demand is approximately 23%-29% above weekend demand. Weekdays show morning and afternoon peaks, whereas weekends have a broader midday profile. Across the two cities and seasons, 16:00-18:00 carries approximately 70%-105% more departures than 07:00-09:00. [3]

Station-level direction can reverse within a day. The repository reports net outflow at Sompasaari in the morning peak and net inflow in the evening; Salmisaarenranta shows the opposite pattern. Pasila railway station has cumulative net flow of -11,014 trips across the two seasons. These identify locations worth investigating, although commuting and leisure explanations remain hypotheses because journey purpose was not observed. [3]

The 2025 daily analysis reports correlations between mean temperature and departures of 0.737 in Helsinki and 0.665 in Espoo. Precipitation correlations are -0.251 and -0.289. These describe 214 seasonal daily observations. Daylight, holidays and seasonal travel behaviour change alongside weather, so the correlations do not isolate a causal weather effect. Nor do they establish weather's incremental forecasting value; that requires a separate experiment using information available when predictions are issued. [3]

From Python outputs to a planning interface

The repository contains a static HTML, CSS and JavaScript dashboard structured for GitHub Pages. Leaflet renders station maps over OpenStreetMap tiles, and Chart.js displays analytical charts. A Python exporter produces derived JSON for station histories, hourly profiles, commute peaks, daily weather, model predictions and evaluation summaries. The browser filters precomputed results; it does not train models or call a live forecasting backend. [6]

Four dashboard areas cover station mapping and ranking, commute dynamics, weather exploration and demand benchmarks. Controls support year, month or selected quarter, municipality and flow metric, alongside station search, details and CSV export. Unmapped stations have a separate audit representation. The top-station heatmap communicates selected patterns, while the wider interface supports inspection beyond the busiest stations. Q4 covers October only because the operating season ends that month.

This structure keeps the interface relatively lightweight, but results remain static until exports are regenerated. External tiles and libraries require network access. Model logic duplicated between notebooks and the exporter can also drift. The repository includes weather, export, deliverable-consistency and UI tests, but the presence of test files alone does not demonstrate that all tests pass in a fresh environment.

What an allocation priority means

Monthly net flow screens for repeated directional imbalance. Actual stock also depends on starting inventory, operator transfers, maintenance and within-day reversals. Completed journeys do not reveal people unable to hire or return a bike. Consequently, high usage or negative monthly flow cannot alone justify a precise capacity increase. Our contribution is a planning prototype and an evidence-based review list; operational recommendations require occupancy and service-level data.

5. Reflection, learning and next steps

Changes from the original canvas

The canvas was useful because it linked technical work to a planning decision. We achieved two-season collection, station-level aggregation, maps, weather exploration, three forecasting benchmarks and an interactive interface. The main compromise was breadth: several forecast horizons, forecast weather and uncertainty were narrowed to monthly benchmarks and historical hourly profiles. Wind, predictive intervals and an optimized allocation policy remain unfinished. The resulting project is a foundation for short-horizon hotspot prediction, rather than a completed real-time prediction service.

The strongest methodological improvement was correcting leakage before comparing models. The most instructive result was that greater model complexity did not improve overall count accuracy. A seasonal baseline gave us a meaningful reference, and disaggregated evaluation revealed weaknesses hidden by network totals. We also learned to connect technical measures carefully to business meaning: a trip is not a physical bicycle, monthly flow is not inventory, and correlation is not a measured intervention effect.

What we would do differently

If starting again, we would define the forecast horizon, decision time and evaluation protocol before building the interface. We would first establish a small reproducible pipeline and baseline, then add complexity in response to measured weaknesses. A shared modelling module would reduce duplication. Explicit timestamp parsing and timezone checks in the common cleaner, duplicate audits, download checksums, a historical station-status panel and versioned capacities would strengthen the data foundation. These improvements would be more valuable than adding another algorithm without improving evaluation.

Future development

The first priority is broader temporal validation across more seasons, choosing hyperparameters on earlier windows and keeping a later season untouched. We would investigate station-scaled or count models and combine same-month history, recent lags and calendar variables. Archived weather forecasts with issue timestamps would enable fair daily or hourly tests of temperature, precipitation and wind. Prediction intervals should be checked for coverage rather than simply displayed.

The next operational step is collecting timestamped occupancy, station availability and redistribution records. These could support empty/full-station risk forecasts and evaluation against service outcomes. Capacity recommendations could then consider lost demand, costs and coverage in quieter neighbourhoods. An operator pilot should assess availability improvements before operational impact is claimed. Publishing aggregate outputs, preserving missingness and retaining source attribution remain essential.

Evidence and reproducibility. Based on repository source and stored results at commit 6022d267a17b3e08654b9e7ef054fdb23dba416e, accessed 23 October 2026. The raw-data pipeline and full test suite were not rerun for this report. Reflections interpret the documented implementation; user-testing and deployment outcomes are not claimed.

References. All paths refer to github.com/luosuo890/HSL_City_Bike_Project.

[1] HSL_City_Bike_Project_Canvas.md. [2] README.md; src/download_data.py; src/clean_and_aggregate.py; src/aggregate_intraday.py; tests/test_weather_integration.py. [3] reports/findings.md. [4] notebooks/model_comparison.ipynb. [5] docs/data/data_model_summary.json. [6] src/export_web_data.py; docs/index.html; docs/js/map.js.